

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «АПО ЭВМиС»

Выполнил:
Студент 4-го курса
Группы АС-66
Головкина В.Д.
Проверил:
Кулик А.Д.

Цель работы: изучение программной модели микропроцессора, регистров, организации памяти; практическая работа с ассемблерными вставками.

Вариант 3: Даны 4 строки. Поменять 1-ю с 3-ей, 2-ю с 4-й.

Ход работы

Задание 1. Изучить следующие теоретические сведения: программная модель микропроцессора; регистры и их назначение; организация памяти; основные конструкции и базовые команды ассемблера (используя материалы лекций и методические указания).

Задание 2. Написать ассемблерную вставку, реализующую обработку строки согласно варианту, выданному преподавателем. Оформить ее в виде отдельной функции.

Задание 3. Реализовать данную обработку строки также в виде функции на C++.

Сравнить быстродействие обоих вариантов. В отчете отразить выводы.

Код :

```
#include <iostream>
```

```
#include <string>
```

```
#include <chrono>
```

```
using namespace std;
```

```
// =====
```

```
// Обмен указателей на строки средствами C++
```

```
// =====
```

```
void swapCpp(string*& str1, string*& str2, string*& str3, string*& str4)
```

```
{
```

```
    string* temp;
```

```
    temp = str1;
```

```
    str1 = str3;
```

```
    str3 = temp;
```

```
    temp = str2;
```

```
    str2 = str4;
```

```
    str4 = temp;
```

```
}
```

```
// =====
```

```
// Обмен указателей на строки средствами ASM
```

```
// GCC inline assembly
```

```
// =====
```

```
void swapAsm(string*& str1, string*& str2, string*& str3, string*& str4)
```

```
{
```

```
string* temp;
```

```
asm volatile(  
    "movq %1, %0"  
    : "=r"(temp)  
    : "r"(str1)  
);
```

```
asm volatile(  
    "movq %1, %0"  
    : "=r"(str1)  
    : "r"(str3)  
);
```

```
asm volatile(  
    "movq %1, %0"  
    : "=r"(str3)  
    : "r"(temp)  
);
```

```
asm volatile(  
    "movq %1, %0"  
    : "=r"(temp)  
    : "r"(str2)  
);
```

```
asm volatile(  
    "movq %1, %0"  
    : "=r"(str2)  
    : "r"(str4)  
);
```

```
asm volatile(  
    "movq %1, %0"  
    : "=r"(str4)  
    : "r"(temp)  
);  
}
```

```
int main()
```

```
{  
    string str1, str2, str3, str4;  
  
    // Ввод четырех строк  
    cout << "Enter 1st string: ";  
    cin >> str1;  
  
    cout << "Enter 2nd string: ";  
    cin >> str2;  
  
    cout << "Enter 3rd string: ";  
    cin >> str3;  
  
    cout << "Enter 4th string: ";  
    cin >> str4;  
  
    // -----  
    // Подготовка двух одинаковых наборов данных  
    // -----  
  
    string cpp1 = str1;  
    string cpp2 = str2;  
    string cpp3 = str3;  
    string cpp4 = str4;  
  
    string asm1 = str1;  
    string asm2 = str2;  
    string asm3 = str3;  
    string asm4 = str4;  
  
    string* cppP1 = &cpp1;  
    string* cppP2 = &cpp2;  
    string* cppP3 = &cpp3;  
    string* cppP4 = &cpp4;  
  
    string* asmP1 = &asm1;  
    string* asmP2 = &asm2;  
    string* asmP3 = &asm3;  
    string* asmP4 = &asm4;
```

```

// =====
// Выполнение C++ версии
// =====

swapCpp(cppP1, cppP2, cppP3, cppP4);

// =====
// Выполнение ASM версии
// =====

swapAsm(asmP1, asmP2, asmP3, asmP4);

// =====
// Вывод результатов
// =====

cout << "\nC++ result:\n";
cout << *cppP1 << endl;
cout << *cppP2 << endl;
cout << *cppP3 << endl;
cout << *cppP4 << endl;

cout << "\nASM result:\n";
cout << *asmP1 << endl;
cout << *asmP2 << endl;
cout << *asmP3 << endl;
cout << *asmP4 << endl;

// Проверка совпадения результатов
if (*cppP1 == *asmP1 &&
    *cppP2 == *asmP2 &&
    *cppP3 == *asmP3 &&
    *cppP4 == *asmP4)
{
    cout << "\nResults are identical.\n";
}
else
{
    cout << "\nResults are different!\n";
}

```

```
// =====  
// Сравнение быстродействия  
// =====
```

```
const int ITERATIONS = 10000000;
```

```
// C++ измерение
```

```
string test1 = str1;
```

```
string test2 = str2;
```

```
string test3 = str3;
```

```
string test4 = str4;
```

```
string* p1 = &test1;
```

```
string* p2 = &test2;
```

```
string* p3 = &test3;
```

```
string* p4 = &test4;
```

```
auto startCpp = chrono::high_resolution_clock::now();
```

```
for (int i = 0; i < ITERATIONS; ++i)
```

```
    swapCpp(p1, p2, p3, p4);
```

```
auto endCpp = chrono::high_resolution_clock::now();
```

```
auto cppTime =
```

```
    chrono::duration_cast<chrono::microseconds>(  
        endCpp - startCpp
```

```
    ).count();
```

```
// ASM измерение
```

```
test1 = str1;
```

```
test2 = str2;
```

```
test3 = str3;
```

```
test4 = str4;
```

```
p1 = &test1;
```

```
p2 = &test2;
```

```
p3 = &test3;
```

```
p4 = &test4;
```

```

auto startAsm = chrono::high_resolution_clock::now();

for (int i = 0; i < ITERATIONS; ++i)
    swapAsm(p1, p2, p3, p4);

auto endAsm = chrono::high_resolution_clock::now();

auto asmTime =
    chrono::duration_cast<chrono::microseconds>(
        endAsm - startAsm
    ).count();

// =====
// Вывод результатов измерения
// =====

cout << "\n===== \n";
cout << "PERFORMANCE COMPARISON\n";
cout << "===== \n";

cout << "Iterations: " << ITERATIONS << endl;
cout << "C++ time:  " << cppTime << " us" << endl;
cout << "ASM time:  " << asmTime << " us" << endl;

if (asmTime > 0)
{
    cout << "C++ / ASM: "
        << static_cast<double>(cppTime) / asmTime
        << endl;
}

return 0;
}

```

```
PS C:\Users\Пип\Documents\4 курс\АПО ЭВМ\1> .\main.exe
Enter 1st string: 1
Enter 2nd string: 2
Enter 3rd string: 3
Enter 4th string: 4

C++ result:
3
4
1
2

ASM result:
3
4
1
2

Results are identical.

=====
PERFORMANCE COMPARISON
=====
Iterations: 10000000
C++ time: 21490 us
ASM time: 19850 us
C++ / ASM: 1.08262
PS C:\Users\Пип\Documents\4 курс\АПО ЭВМ\1> |
```

Вывод: В ходе лабораторной работы я изучила программные модели микропроцессора, регистров, организации памяти и реализовала практическую работу с ассемблерными вставками.